Technical Design Document: AI-Powered Family Location Tracker

1. Technology Stack

Layer	Technology Choice	Details & Constraints
Frontend / Mobile App	React Native / Flutter	Open-source, cross-platform support for iOS and Android
Map Interface	OpenStreetMap (OSM)	Integrated via `react-native-maps` or `flutter_map`
Backend Framework	Node.js with Express	Event-driven architecture for handling spatial streams
Real-Time Engine	Socket.io	Bidirectional WebSocket communication for live telemetry
Database	MongoDB Atlas	M0 Free Cluster tier (512 MB storage limit)
AI Integration	Google Gen AI SDK	Gemini API via Google AI Studio
Hosting Platform	Render.com / Railway	Free Web Service Tier deployment target

2. System Architecture and Data Flow

2.1 Data Capture Flow

1. **Device Capture:** Mobile app polls device GPS hardware to retrieve precise coordinates locally.
2. **Ingress:** App transmits coordinate payloads to the Node.js backend server via established Socket.io WebSocket connections.
3. **Persistence:** Server asynchronously writes incoming coordinate records to the MongoDB Atlas cluster.

2.2 Real-Time Sync Flow

1. **Event Trigger:** Upon receiving a new location payload through Socket.io, the server initiates an in-memory broadcast process.
2. **Distribution:** The payload is immediately routed to all active, authenticated Socket.io client connections subscribed to the sender's specific family group ID.

2.3 AI Query Flow

1. **Client Request:** User submits a natural language question via the app interface, sending an HTTP `POST` request to the backend.
2. **Context Retrieval:** Backend queries MongoDB Atlas for the relevant target user's 24-hour location trajectory.
3. **Prompt Construction:** Backend dynamically structures a contextual prompt merging the user's explicit question with structured JSON location history.
4. **Inference Call:** Backend issues a request to Google AI Studio using the Gemini API.
5. **Response Delivery:** Gemini returns a natural language summary, which the backend forwards directly to the requesting client app.

3. Database Schema

Collection: `Users`

Stores account profiles and maps user relationships within family units.

- `_id`: `ObjectId` — Primary key.

- `name`: `String` — Full display name of the user.
- `email`: `String` — Unique user email address used for login.
- `passwordHash`: `String` — Securely hashed account password.
- `familyGroupId`: `ObjectId` — Foreign key referencing the user's associated family group.

Collection: `Locations`

Stores time-series geospatial tracking points generated by client devices.

- `_id`: `ObjectId` — Primary key.
- `userId`: `ObjectId` — Foreign key referencing the associated `Users` document.
- `latitude`: `Number` — WGS 84 latitude coordinate in decimal degrees.
- `longitude`: `Number` — WGS 84 longitude coordinate in decimal degrees.
- `timestamp`: `Date` — UTC generation timestamp (indexed for range queries and TTL expiry).

4. API Specifications

HTTP REST Endpoints

Authentication Login

- **Route:** `POST /api/auth/login`
- **Description:** Validates user credentials and issues an authentication session token.
- **Output:** JSON object containing a Signed JWT.

AI Natural Language Location Query

- **Route:** `POST /api/ai/query`
- **Payload:**{

"prompt": "Where did John go today?"

}

- **Response:**{

"answer": "John visited the pharmacy at 4 PM."

}

WebSocket Protocol (Socket.io)

- **Event Name:** `location_update`
- **Direction:** Client to Server (Emitted), Server to Room (Broadcasted)
- **Payload:** `{ "lat": Number, "lng": Number, "timestamp": String }`
- **Behavior:** Updates server-side state and broadcasts spatial coordinates to all connected peers within the same family room context.

5. Authentication, Security, and Integrations

5.1 Authentication & Secrets

- Standard JSON Web Tokens (JWT) generated by Node.js handle session stateless authentication.
- Google AI Studio API Keys and MongoDB URIs are managed strictly as server-side Environment Variables on Render.com and are **never** exposed to client applications.

5.2 Security & Rate Limiting

- To maintain compliance with Google AI Studio's free tier, strict **Rate Limiting** is enforced on the `/api/ai/query` endpoint (capped to a maximum threshold below 15 Requests-Per-Minute).

5.3 Data Lifecycle & Capacity Management

- A MongoDB **Time-To-Live (TTL) index** is applied to the `Locations.timestamp` field.
- Records are set to automatically expire and purge after **30 days**, guaranteeing that spatial data overhead remains well within the 512 MB free tier database limit.